NoSQL

Not only SQL (Meaning don't have SQL for NoSQL DBs)

or

Non-Relational DBs

SQL	NoSQL
RDBMS	Non Relational DBs

※ Best if you want to scale DBs more better than SQL DBs

1) Key-Value: stores — Eg:- Redis, Memcached, etcd

eg id → object

KV map

Benefit:- It the Read / Write can be quick as it uses RAM used if you want to read or write a large amount of data

2) Document Store:

COLLECTION OF DOCUMENTS

```
{"field":{
    "onetype":[
        {"id":1,"name"...},
        {"id":2,"name"...}
    ],
    "othertype":{"id":2,...}
},
"field2":{
    ..
}
}
```

JSON

Benefit:- In document store is more flexible meaning we need not define a schema like SQL DBs (mix match types or add/remove fields)

eg:- MongoDB (open source)

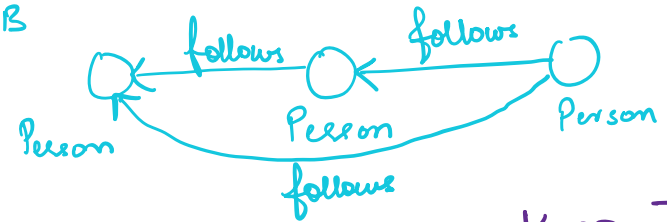
3) Wide-column DBs:

- Can handle massive scale but they are typically used to store a lot of writes like Time series data.
  Also, provides flexibility — No schema. — Schema can be added
- Also better to use wide-column DBs when we do not want to read/update a lot

eg:- Cassandra, Big Table

4) Graph Based DBs: These are about Relations like in case of Social Media we want to store the info of who follows whom or secondary relations (mutual friends) So this kind of info can be stored in graphs more importantly like a directed graph

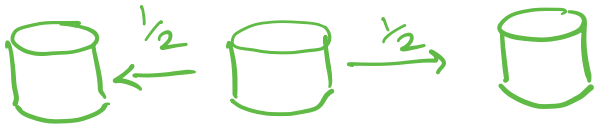
eg:- Neo4j, Cosmos DB etc



- X. Graph DBs are built on SQL, keep the relational aspect of DB's so they are not entirely NoSQL DBs

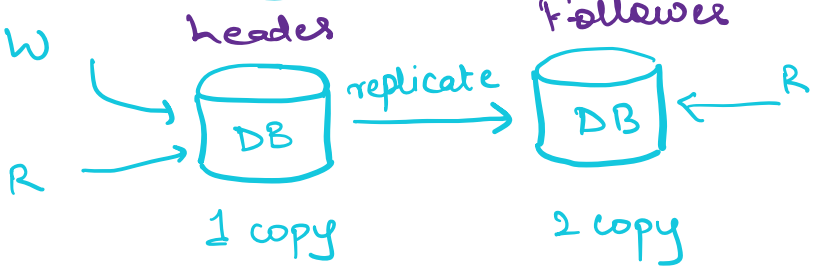
X. The major benefit of NoSQL is it allows us to scale Data / DB Horizontally something that cannot be achieved in SQL where vertical scaling can be done but Horizontal scaling is difficult due to ACID property where constraints like Foreign Key need to be managed. Whereas NoSQL dont necessarily worry about ACID properties

NoSQL (Allows to split DBs)



BaSE (NoSQL acronym)

=> Eventual Consistency



2 copies of Data in NoSQL

Everytime we write he will write to the header DB and the header DB will be responsible to replicate the data to the follower DB. However in this case if some people read data from Follower they will get stale data / not up to date data for some time which is ok as eventual consistency will be achieved after write

probably in ms or seconds

eg:- Used in case of Twitter follower count where the number of follower count won't matter

Twitter

↑ Read Capacity

= Almost same Write Capacity

- We can split DB in SQL DBs which is called as Sharding but again can get very complicated unlike NoSQL where splitting of data is readily available